FUNCTIONAL REQUIREMENTS DOCUMENT

E-Commerce UI Test Automation Framework

Project: Playwright + TypeScript Automation Framework
Application Under Test: AutomationExercise
GitHub: paritk7210-tester/TypeScript
Version: 1.0 **Date:** August 12, 2026
Prepared By: Parit Katoch — Business Analyst / Test Analyst
Status: Portfolio Project

1. Purpose

This Functional Requirements Document translates the business requirements for the Playwright + TypeScript e-commerce automation project into detailed functional behavior, validation rules and testable acceptance criteria. It is intended to demonstrate the BA flow from business requirement to functional requirement and automated validation.

2. System Overview

The solution is a UI test automation framework that validates customer-facing e-commerce journeys in AutomationExercise. Playwright provides browser automation and assertions, while TypeScript provides the implementation language. The framework uses Page Object Model, reusable fixtures, common utilities and JSON-based test data.

3. Functional Architecture

The framework is organized around reusable page classes, fixtures, utilities, test data and test specifications. Page objects encapsulate UI interactions; fixtures provide reusable test setup; utilities provide common actions/data handling; JSON files centralize applicable test data; test specifications validate business journeys.

4. Functional Requirements

ID	Module	Functional Requirement	Acceptance Criteria
FR-01	Registration	The system shall support customer registration using the available signup flow.	User can navigate to signup, submit valid details and reach the expected registered-user state. Validation is captured for invalid/duplicate inputs where applicable.
FR-02	Login	The system shall authenticate users through the login flow.	Valid credentials allow access to the expected authenticated state; invalid credentials are rejected with the expected application response.
FR-03	Home / Navigation	The system shall allow users to navigate between major e-commerce sections.	Expected navigation links/buttons open the corresponding pages and URLs/page states are validated.
FR-04	Products	The system shall allow users to browse/search products and open product details.	Product listing is displayed; applicable search results are shown; selected product details can be opened and validated.
FR-05	Cart	The system shall allow users to add products and manage cart contents.	Selected product appears in cart; quantity/item details can be validated; cart state is retained during the tested journey.
FR-06	Checkout	The system shall allow a user to proceed from cart through checkout.	Checkout page loads; required customer/order information is displayed or entered; user can progress through the expected checkout sequence.
FR-07	Payment	The system shall validate successful progression to the payment page without performing a real transaction.	After checkout progression, the expected payment-page URL/state is verified.

ID	Module	Functional Requirement	Acceptance Criteria
FR-08	Contact	The system shall support the customer contact form.	User can enter contact information/message, submit the form and validate the expected response/confirmation.
FR-09	File Upload	The system shall support file upload through the contact journey where provided by the application.	A valid test file can be attached and the submission flow completes with the expected application behavior.
FR-10	Subscription	The system shall support subscription registration from the available subscription component.	User can enter subscription information and the expected success/response state is validated.
FR-11	Test Cases	The system shall allow navigation to and validation of the Test Cases section.	Test Cases page opens and the expected page state/content is validated.
FR-12	Cross Browser	The framework shall execute applicable tests across Chromium, Edge and Firefox.	Configured browser projects execute the selected test suite and produce individual execution results.
FR-13	Test Data	The framework shall support centralized JSON test data.	Tests can read applicable data from JSON files rather than duplicating test values throughout specifications.
FR-14	Framework Reuse	The framework shall use Page Object Model, fixtures and common utilities.	Common actions and page interactions are reusable and changes to a page locator can be managed centrally.
FR-15	Reporting	The framework shall provide execution evidence for test results.	HTML report is generated; screenshots, videos and traces are available according to configured execution/reporting settings.

5. Detailed Functional Flow

Flow A — Registration: Home → Signup/Login → Enter registration data → Submit → Validate account/response.

Flow B — Login: Login → Enter credentials → Submit → Validate authenticated state → Logout where applicable.

Flow C — Product to Cart: Products → Search/Browse → Product Details → Add to Cart → Cart → Validate item/quantity.

Flow D — Checkout: Cart → Checkout → Enter/confirm required details → Payment → Validate payment-page state.

Flow E — Contact: Contact Us → Enter form details → Upload test file where applicable → Submit → Validate response.

Flow F — Subscription: Subscription section → Enter email → Submit → Validate subscription response.

6. Validation Rules

• Assertions must validate business outcomes or meaningful page states rather than only checking that a click occurred.

• Locators should be maintained within page objects to reduce duplication.

• Tests should use Playwright synchronization and assertions instead of unnecessary fixed waits.

• Test data should be separated from test logic where data-driven execution is appropriate.

• Checkout/payment automation must not use real payment credentials or perform a real financial transaction.

• Failures should retain sufficient diagnostic evidence to support defect analysis.

7. Error / Exception Handling

Scenario	Expected Handling
Invalid login	Application validation/error response is captured and test verifies that authentication does not succeed.
Invalid/duplicate registration	Application validation is captured and test verifies expected behavior.
Missing required checkout information	User remains on the relevant step or receives validation according to application behavior.
Product/cart mismatch	Test fails with assertion evidence identifying expected versus actual state.

Scenario	Expected Handling
Page timeout / unavailable application	Execution captures timeout/error evidence and distinguishes environment failure from application defect.
Cross-browser failure	Failure is recorded against the specific browser project for investigation.

8. Traceability Matrix

BR	Functional Coverage	Automation Evidence
BR-01	Registration / authentication	Registration and login automated scenarios
BR-02	Product discovery	Products/search/product-detail scenarios
BR-03	Cart management	Cart scenarios
BR-04	Checkout journey	Checkout scenario(s)
BR-05	Payment-page validation	Payment-page URL/state assertion
BR-06	Reusable automation	POM, fixtures and utility framework
BR-07	Cross-browser	Chromium, Edge and Firefox configurations
BR-08	Centralized test data	JSON test-data files
BR-09	Execution evidence	HTML report, screenshots/video/traces where enabled
BR-10	Framework maintainability	Page classes, fixtures and common actions

9. Test Environment & Dependencies

The project is designed for execution from the TypeScript/Playwright project environment with Node.js, npm, TypeScript and Playwright installed. Browser projects include Chromium, Edge and Firefox. The AutomationExercise web application is the application under test. Git/GitHub is used for source control and portfolio delivery.

10. Non-Functional Functional-Test Considerations

The framework should remain maintainable, reusable, reliable and diagnosable. Cross-browser execution, centralized test data, stable selectors, appropriate waits/assertions and execution evidence are key quality considerations.

11. Defect Management

When an automated scenario fails, the failure should be analyzed to determine whether the cause is an application defect, automation defect, test-data issue, browser-specific behavior or environment issue. Reproducible application defects should be documented with steps, expected result, actual result, severity/priority, browser/environment and supporting evidence.

12. Definition of Done

• Functional requirement has corresponding test coverage.

• Automated scenario passes in the intended browser configuration.

• Expected business outcome is asserted.

• Test data and reusable components are maintained in the appropriate framework layer.

• Execution evidence is available for failures and diagnostic scenarios.

• Any identified application defect is documented and retested after resolution.

13. Document Approval

This FRD is a portfolio-level functional specification derived from the project's BA BRD. In a production project, the FRD would be reviewed and baselined by Product/Business, Engineering and QA stakeholders before implementation or formal test sign-off.